

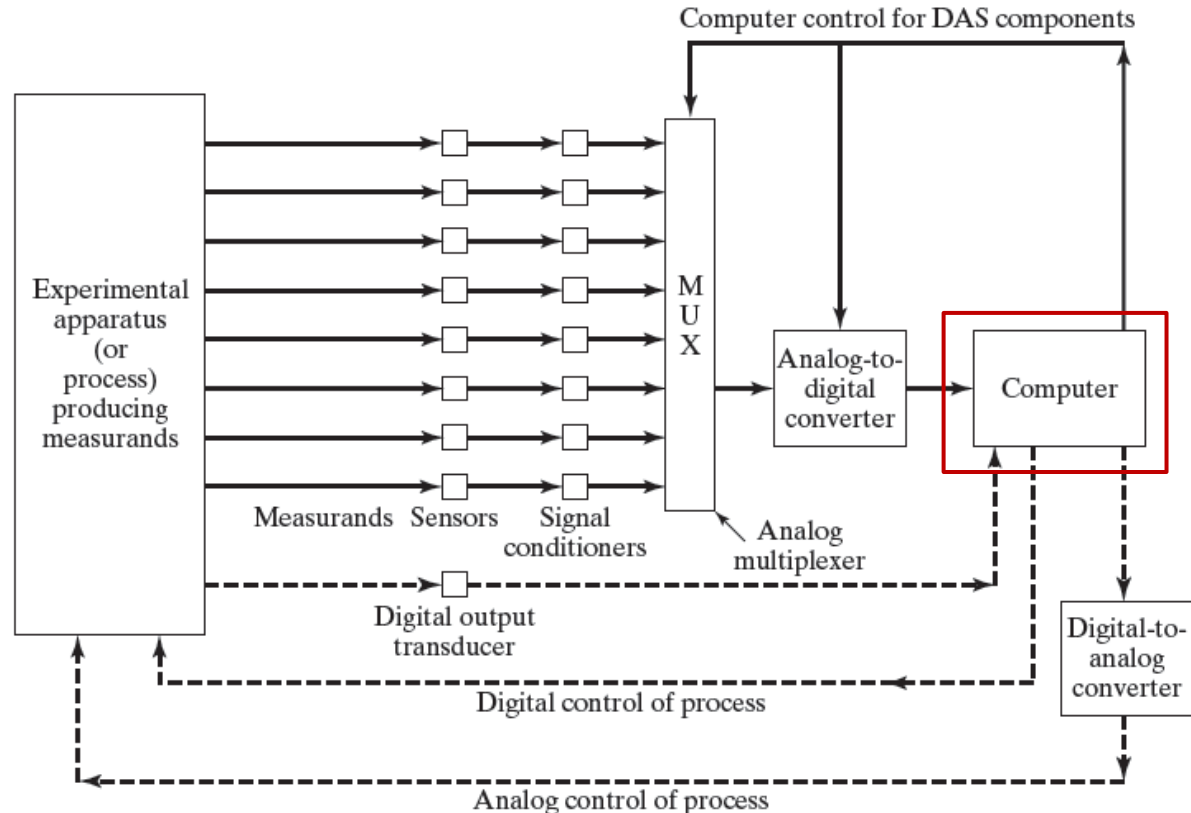
ELT3207 Meas. & Digital Control Basics

Lecture 5: I/O with Computers

Hoang Gia Hung

Faculty of Electronics and Telecommunications
University of Engineering and Technology, VNU Hanoi

Introduction



□ Today's lecture: How the computer communicate with the DAQ

Computer Organization

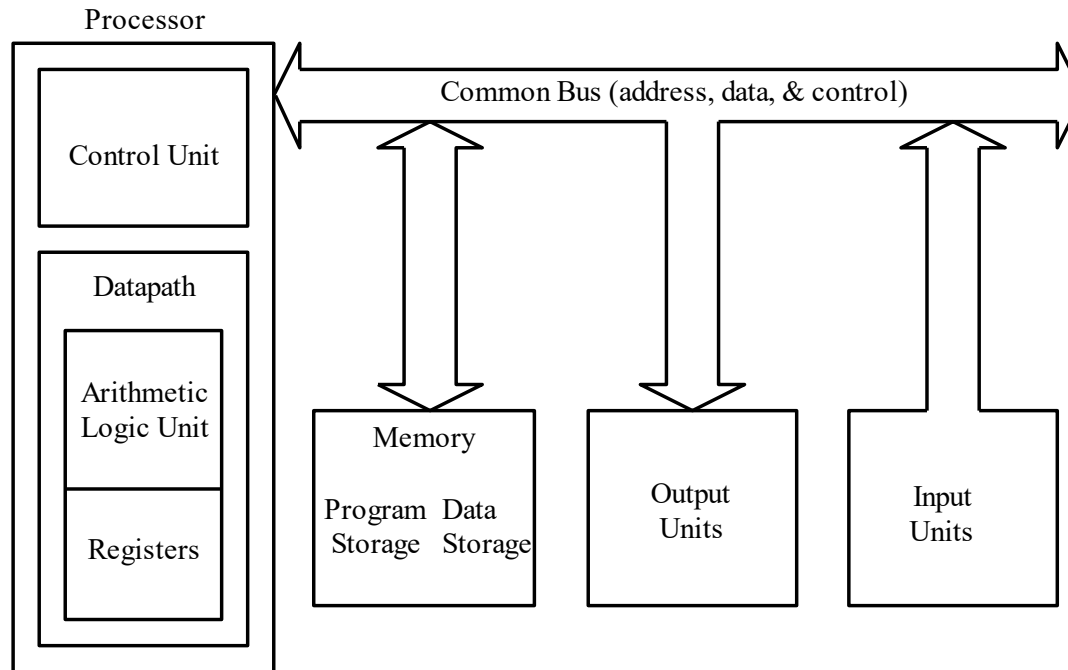
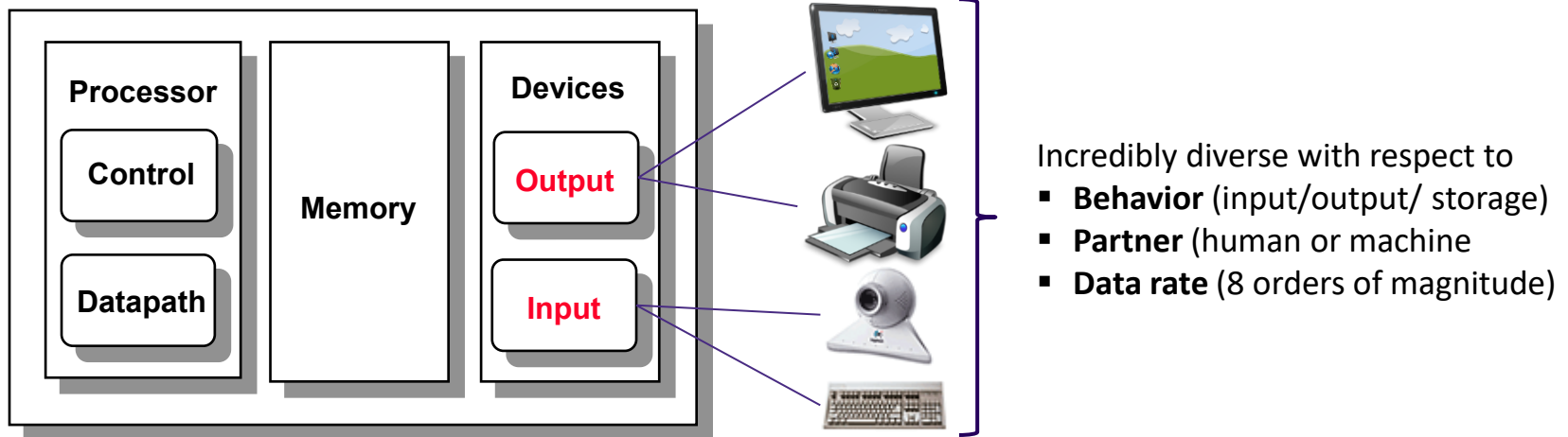


Figure 1.1 Computer Organization

- ❑ Computers interface with the outside devices via their I/O Units

I/O devices



□ Key Questions:

- How do we control diverse devices with vastly different speeds?

Interconnect hierarchy

- How do we share I/O resources fairly among programs? Bus arbitration

- How does a program interact with the outside world?

Special I/O instructions, Memory-mapped I/O

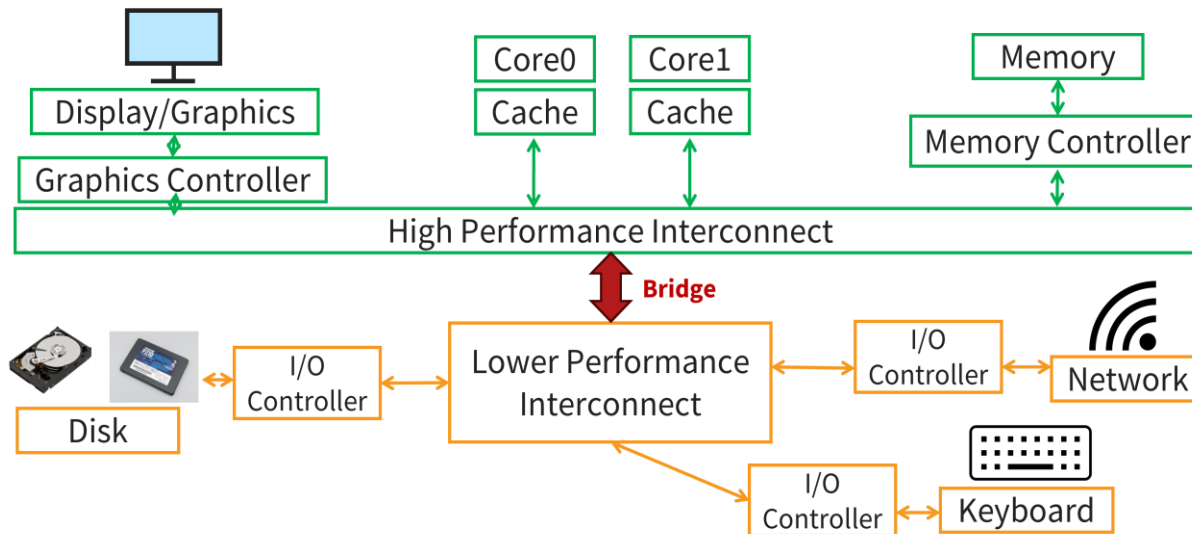
- How do we maintain performance while handling I/O? Interrupt, DMA

Processor-I/O Speed Mismatch

- ❑ Analogy: The CPU is the brain; I/O is the body
 - Brain: 1 GHz processor I/O throughput = 4 GB/s (load/store instructions)
 - Body: Typical I/O devices are much slower
 - Keyboard: 10 B/s → 400 million times slower
 - USB 3.1: 1.25 GB/s → 3x slower
 - Fast SSD: 560 MB/s → 7x slower
 - These are peak rates – actual throughput is lower
 - Common I/O devices neither deliver nor accept data at processor speed
- ❑ Challenge: Find solutions to bridge this gap efficiently
 - System architectures
 - Communication protocols
 - System integration

Interconnect Hierarchy

- Two distinct interconnects: Different speeds for different needs
 - North side bus (fast) & South side bus (slow) are joined by a bridge



- Why it works:
 - Physics & cost: Keep the north side bus short and close to the CPU is efficient
 - Concurrency: Fast communication is not bottlenecked by slower one
 - Flexibility: Allows a wide variety of peripherals to be attached

Buses

❑ Set of wires that connect subsystems together

- Act as a shared communication link between subsystems
- Need to support a wide range of devices, latencies, transfer rates
- The maximum bus speed is limited by bus length & number of devices on it

❑ Advantages:

- Versatile: Easy to add new devices
- Portable: Devices work on different systems
- Low cost: Single set of wires shared

❑ Disadvantages:

- Bottleneck: Bandwidth limits throughput
- Length limited: Signal degradation
- Speed limited: By slowest/farthest device

Buses - Motherboard Buses



Buses - Connection Cables

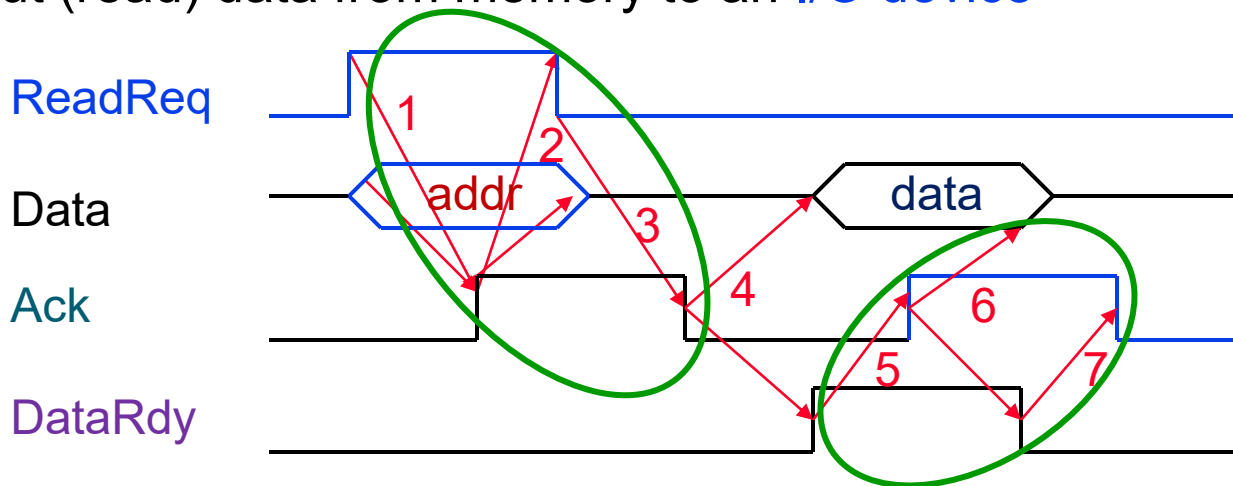


Synchronous and Asynchronous Buses

- ❑ Synchronous bus (e.g., processor-memory buses)
 - Includes a clock in the control lines and has a fixed protocol for communication that is relative to the clock
 - Advantage: involves very little logic and can run very fast
 - Disadvantages: (1) Every device communicating on the bus must use same clock rate; (2) To avoid clock skew, they cannot be long if they are fast
- ❑ Asynchronous bus (e.g., I/O buses)
 - It is not clocked, so requires a handshaking protocol and additional control lines (ReadReq, Ack, DataRdy)
 - Advantages: (1) Can accommodate a wide range of devices and device speeds; (2) Can be lengthened without worrying about clock skew or synchronization problems
 - Disadvantage: slow(er)

Asynchronous Handshaking Protocol

❑ Output (read) data from memory to an I/O device



I/O device signals a request by raising **ReadReq** and putting the **addr** on the data lines

1. Memory sees **ReadReq**, reads **addr** from data lines, and raises **Ack**
2. I/O device sees **Ack** and releases the **ReadReq** and data lines
3. Memory sees **ReadReq** go low and drops **Ack**
4. When memory has **data** ready, it places it on data lines and raises **DataRdy**
5. I/O device sees **DataRdy**, reads the **data** from data lines, and raises **Ack**
6. Memory sees **Ack**, releases the data lines, and drops **DataRdy**
7. I/O device sees **DataRdy** go low and drops **Ack**

Modern Bus Technologies

- ❑ Processor-memory bus (“Front Side Bus”, proprietary)
 - Short and high speed
 - Matched to the memory system to maximize the memory-processor bandwidth
 - Optimized for cache block transfers
- ❑ I/O bus (industry standard, e.g., SCSI, USB, Firewire)
 - Usually is lengthy and slower
 - Needs to accommodate a wide range of I/O devices
 - Use either the processor-memory bus or a backplane bus to connect to memory
- ❑ Backplane bus (industry standard, e.g., ATA, PCIe)
 - Allow processor, memory and I/O devices to coexist on a single bus
 - Used as an intermediary bus connecting I/O busses to the processor-memory bus

Bus Arbitration

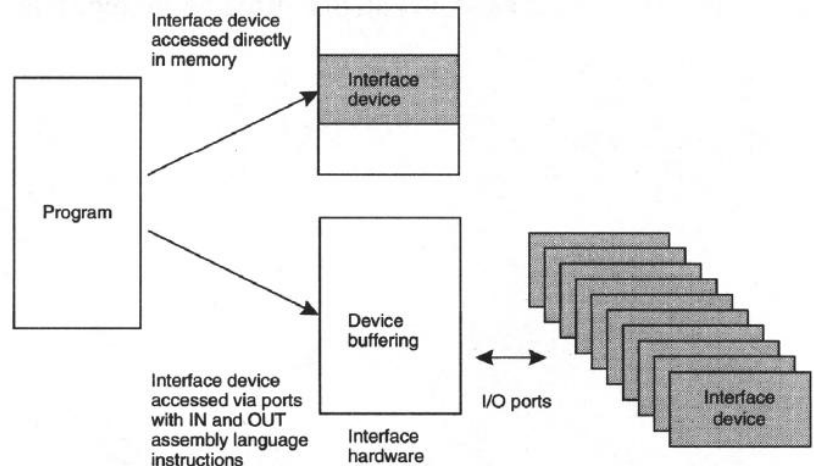
- ❑ The problem: Multiple devices may need the bus simultaneously
 - How do we decide who gets it, taking into account priority & fairness?
- ❑ Daisy chain
 - Arbiter → Dev1 → Dev2 → ... → DevN (priority decreases down the chain)
 - Very simple but unfair & slow
- ❑ Centralized parallel: Most common in modern systems
 - Each device has dedicated request/grant lines to a central arbiter
 - Flexible, fair but complex arbiter
- ❑ Other schemes:
 - Distributed: Consensus-based selection of the highest-priority requester
 - Every device knows its priority & the state of the shared arbitration line
 - Each device compares its request against others, withdraws if loses priority
 - Collision detection: Use bus when free; retry on collision (Ethernet)

Accessing I/O Devices

- ❑ How does a program access a specific I/O device?
 - Must go through the corresponding I/O controller module
 - Each I/O controller is assigned a unique address/set of addresses

- ❑ Memory-Mapped I/O
 - Memory address assigned to I/O
 - Use regular load/store
 - OS controls I/O address access
 - Used by: RISC-V, ARM

- ❑ Special I/O Instructions
 - Dedicated IN/OUT instructions
 - Specify device and command
 - Separate I/O address space
 - Used by: x86 (historical)



Functions of I/O controllers

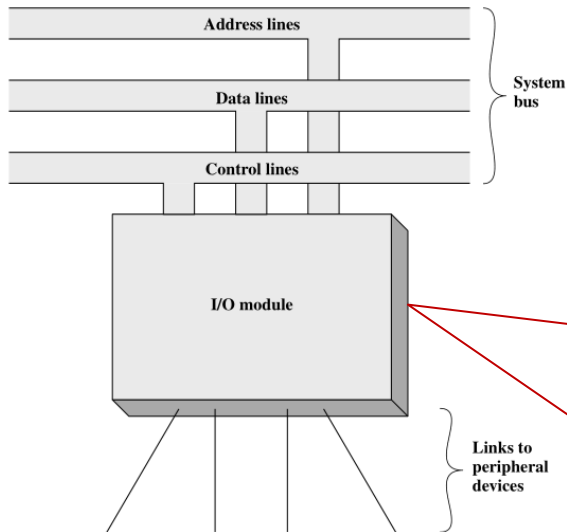


Figure 7.1 Generic Model of an I/O Module

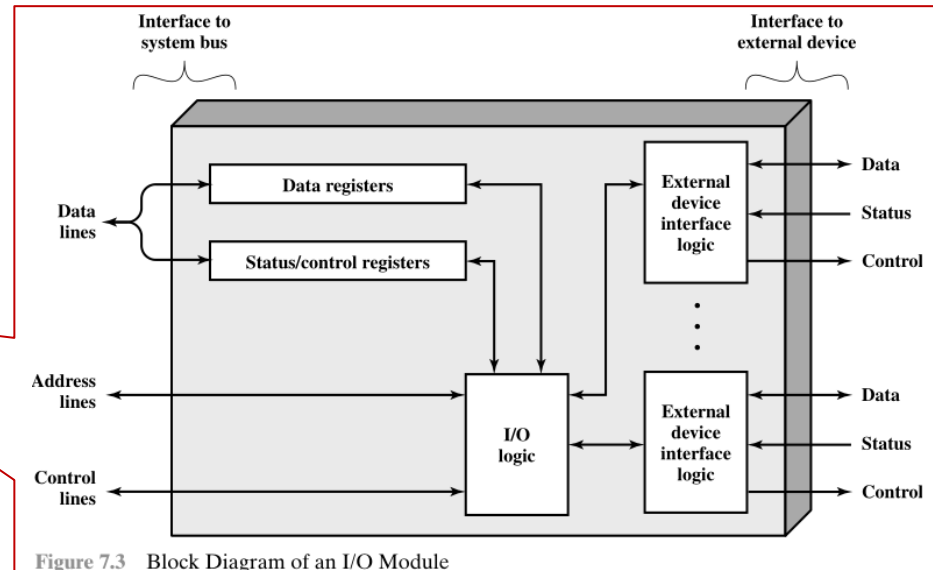
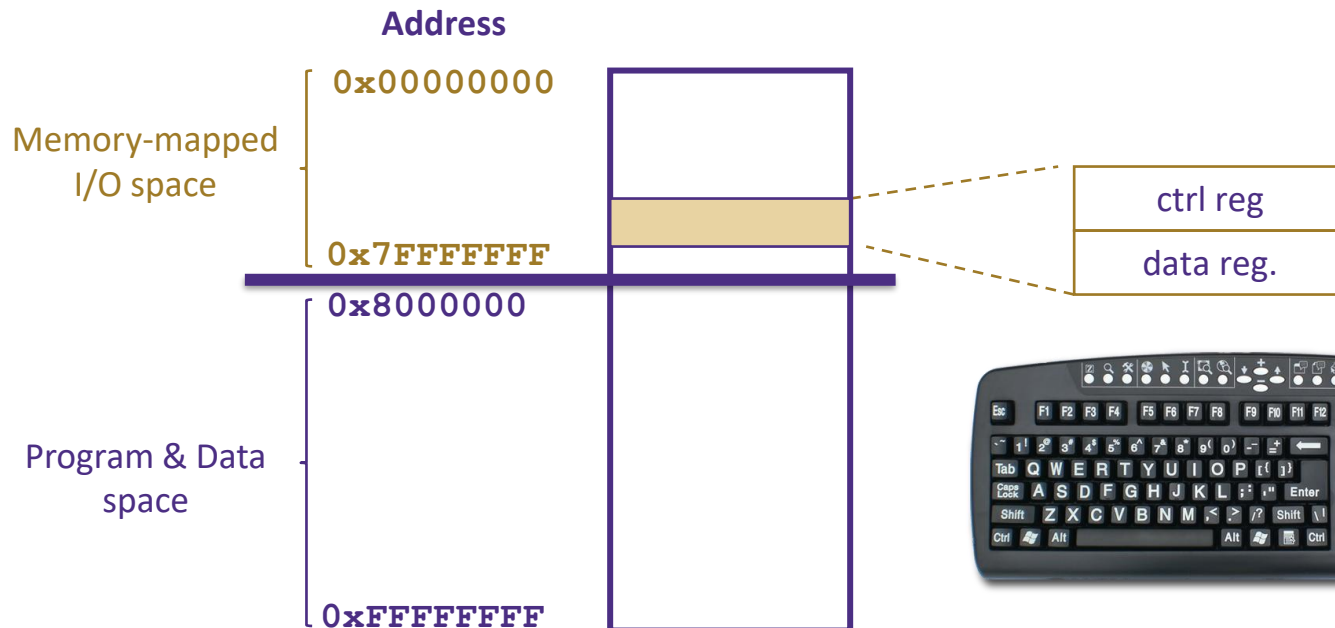


Figure 7.3 Block Diagram of an I/O Module

- ❑ Major functions of an I/O controller (e.g. RS-232, USB, ...) are:
- Command decoding
 - Data buffering: via data registers (memory data rate $\gg$ peripheral data rate)
 - Status reporting (BUSY/READY): via status registers (may also function as a control register) to accept detailed control information from the processor.
 - Address recognition: recognizes & generates addresses of the I/O devices.

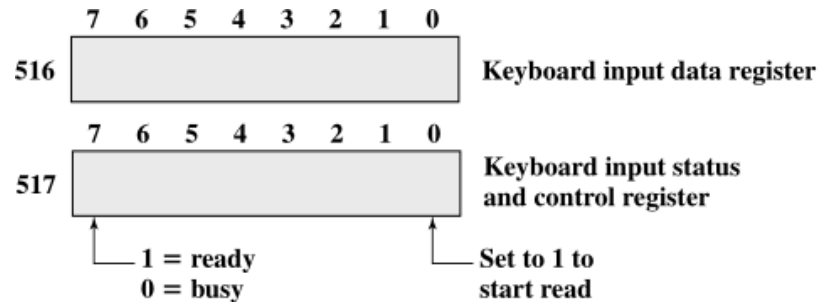
Memory Mapped I/O

- ❑ Certain addresses are not 'regular memory'
 - Instead, they correspond to registers in I/O devices
 - Communicating with devices is performed via regular load/store instructions



Isolated I/O

- ❑ I/O ports are accessible only by a few **special** I/O commands
 - C.f. regular instructions in memory-mapped systems
 - **Example:** Read 1 byte of data from the keyboard into an accumulator



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

(b) Isolated I/O

I/O Protocols: Polling

❑ Concept:

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

- CPU busy-waits in a loop until the device is ready

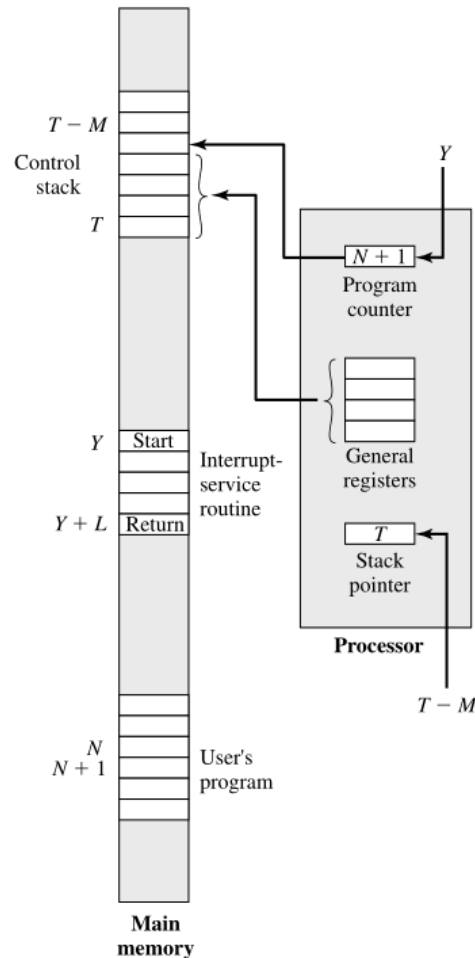
❑ Analysis: 1 GHz processor, polling overhead: 400 cycles

- **Scenario 1:** Mouse I/O - poll 30 times/second
 - $\text{Cycles/sec} = 30 \text{ polls/s} \times 400 \text{ cycles/poll} = 12,000 \text{ cycles/s}$
 - $\% \text{ CPU} = 12,000 / 1,000,000,000 = 0.0012\%$
- **Scenario 2:** HDD transfer - transfer rate 16 MB/s, polling every 16 bytes
 - $\text{Polls/sec} = 16 \text{ MB/s} \div 16 \text{ B/poll} = 1,000,000 \text{ polls/s}$
 - $\text{Cycles/sec} = 1\text{M polls/s} \times 400 \text{ cycles/poll} = 400\text{M cycles/s}$
 - $\% \text{ CPU} = 400,000,000 / 1,000,000,000 = 40\%$
- **Verdict:** Good for low-rate devices, bad for high-rate devices.

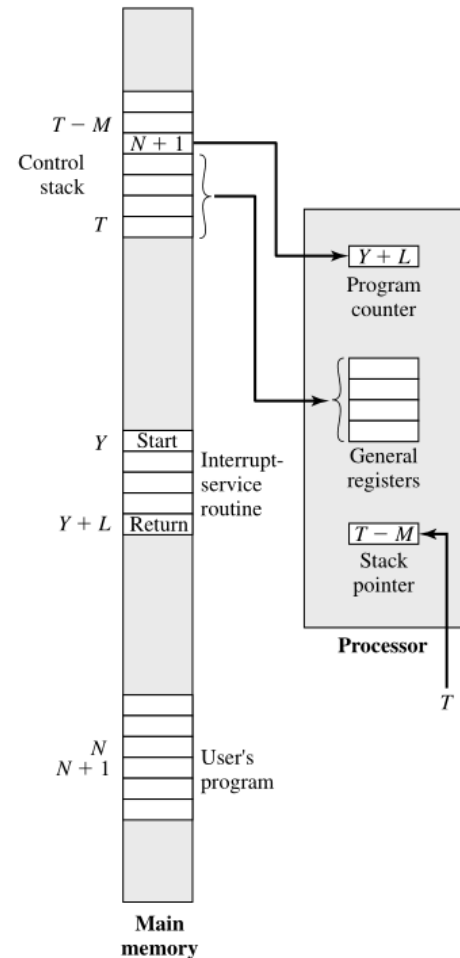
I/O Method: Interrupts

- ❑ Concept: Polling is akin to waiting at the door for guests
 - What about a bell?
- ❑ Computer lingo for bell: Interrupt when I/O needs attention
 1. Normal execution: Program runs normally on CPU
 2. I/O device signals interrupt when it has data or needs service
 3. CPU finishes current instruction, saves state
 4. Jump to interrupt handler (trap to OS)
 5. OS reads/writes data from/to device
 6. Resume: Restore saved state, return to program
- ❑ Interrupt characteristics:
 - **Asynchronous**: Not tied to any specific instruction
 - Can be **prioritized**: High-priority interrupts serviced first
 - Requires **device identification**: Which device interrupted?

Interrupt Processing



(a) Interrupt occurs after instruction at location N



(b) Return from interrupt

Interrupts - Pros and Cons

❑ Advantages:

- CPU doesn't waste time polling
- User program only suspended during actual data transfer
- Efficient for bursty I/O
- Handles multiple devices well
- Supports priority levels

❑ Disadvantages:

- Requires special hardware in both CPU and device
- Overhead of Interrupt Service Routine (ISR)
- Cache/TLB pollution from handler
- Can thrash with very high I/O rates

Interrupt Thrashing at High Sample Rates

❑ High-speed ADC example

- ADC rate: 1 MS/s, one interrupt per sample
- ISR cost: 1,000 cycles/sample
- Interrupt overhead = 1 Gcycles/s → 100% on one 1 GHz core

❑ Why this is a problem

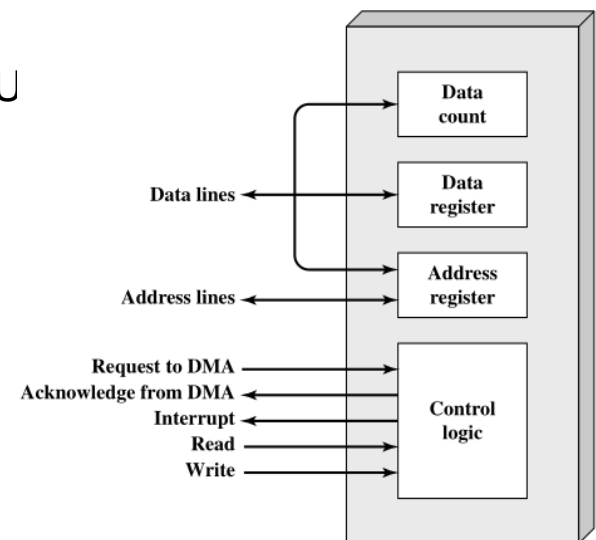
- CPU is overwhelmed by interrupt handling
- Main processing cannot keep up
- System latency and jitter increase (due to disruption at the OS scheduler)
- Overall throughput becomes limited by interrupt overhead

❑ Solution

- Use DMA (Direct Memory Access) with block transfers
- Interrupt once per buffer completion
- Rule of thumb: consider DMA when I/O rate reaches 50 kS/s

Direct Memory Access (DMA)

- ❑ Motivational example: Disk transfer - high sustained rate I/O
 - 16 MB/s with interrupts every 16 bytes → 1 million interrupts per second
 - Overhead for each interrupt: save state, handle transfer, restore state
 - Verdict: Most of CPU time spent on interrupt handling!
- ❑ DMA: I/O devices can read/write the main memory directly
 - Requires new hardware - The DMA Engine
 - DMA engine contains registers written by CPU
 - Memory address: Where to place/get data
 - Data count: How much to transfer
 - Device number: Which I/O device
 - Direction: Read/write
 - Transfer unit: Bytes per burst



How DMA Works

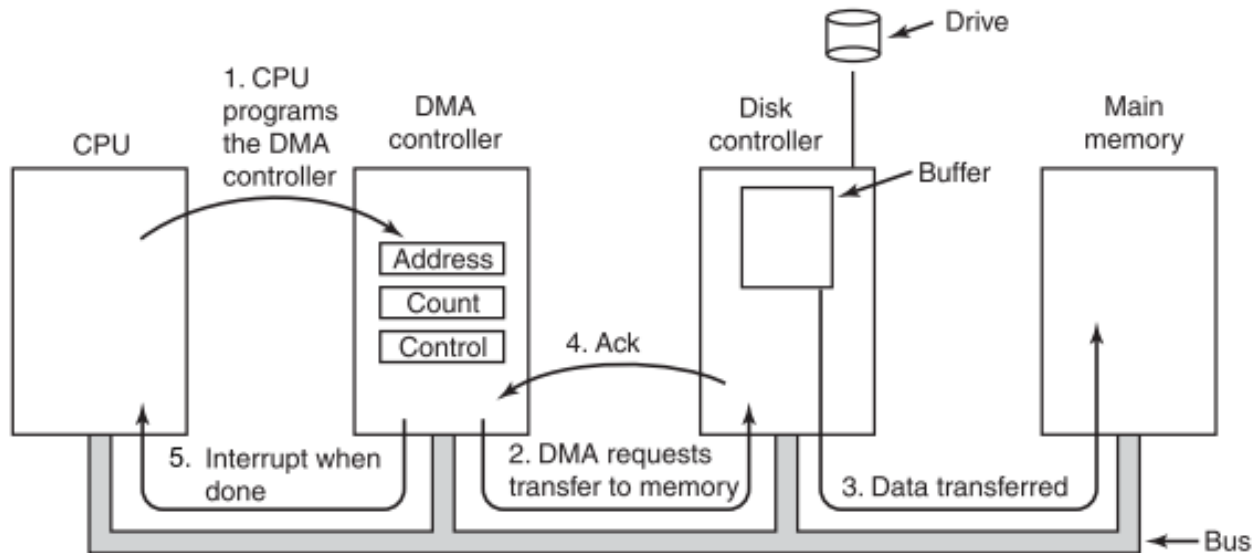


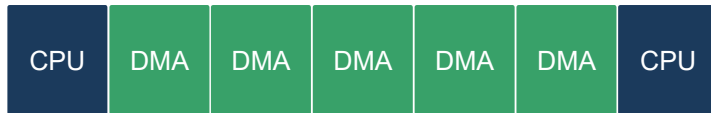
Figure 5-4. Operation of a DMA transfer.

❑ Key benefits

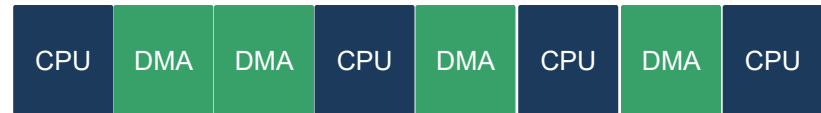
- Offloads data transfer work from the CPU
- Higher I/O throughput
- Reduced interrupt overhead

DMA Modes: Burst vs Cycle-Stealing

Burst Mode



Cycle-Stealing Mode



- ❑ **Burst:** *DMA transfers a contiguous block once it wins bus access*
 - Higher bus efficiency and DMA throughput
 - CPU or other bus masters may experience longer access latency
 - Best for bulk data movement
- ❑ **Cycle-stealing:** *DMA performs small transfers intermittently*
 - CPU and DMA access the bus in a more interleaved fashion
 - Lower peak DMA throughput, but better CPU responsiveness
 - Useful when latency and sharing matter more than raw transfer speed
- ❑ **Note:** These are simplified conceptual modes
 - Modern systems often use configurable burst sizes and arbitration policies.

Summary of I/O methods

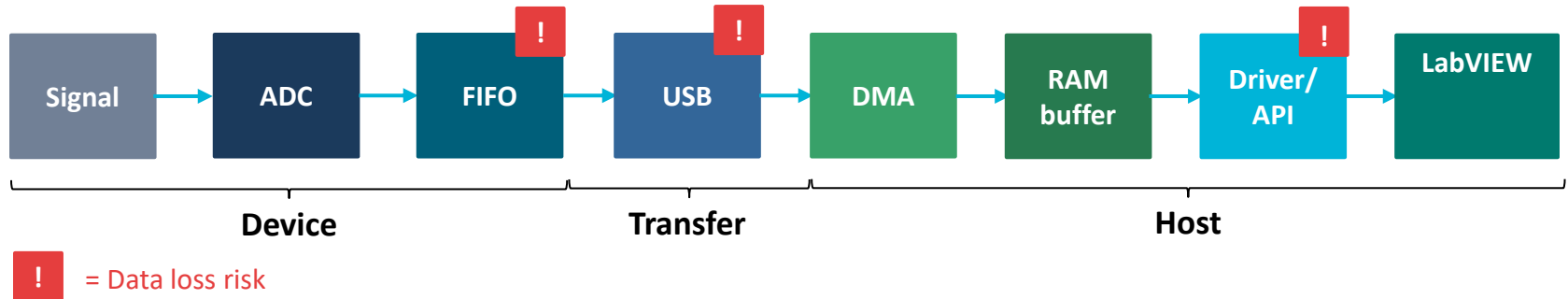
❑ When to use what?

Scenario	Best Method	Reason
Low data rate (keyboard, mouse)	Interrupts	Low overhead, event-driven
Moderate data rate (network)	Interrupts	Only active when data arrives
High sustained rate (disk transfers/GPU)	DMA	Bypassing the CPU entirely to prevent a bottleneck
Real-time constraints	Polling	Predictable timing

❑ Modern systems often use a hybrid approach

- 10Gbps Ethernet: starts with interrupt-driven I/O and switch to polling when the traffic hits a certain threshold - called NAPI (New API) in Linux.
- USB Controller: uses interrupts for the "setup" and "detection" phases but hands the heavy lifting over to DMA for the actual data transfer.

DAQ Data Path and Risks



❑ Potential data loss

- **FIFO overflow:** source produces faster than the system drains
- **USB service gap:** transfer delay causes backpressure
- **Host buffer overrun:** application consumes slower than DMA/driver delivers

❑ Design rule

- Match end-to-end throughput
- Add enough buffer depth to absorb burstiness and software latency

Key takeaways

- ❑ Computer organization for I/O interfacing
 - I/O system microarchitecture
 - Buses
 - Memory-mapped vs. Isolated I/O
- ❑ Common I/O methods
 - Polling
 - Interrupt
 - DMA
 - **Self-study**: Dr. Pham Duy Hung's slides (in Vietnamese).
- ❑ **Next lecture**: Statistical analysis of measurement data